OAuth Approval Guide Agent

APM Integrations Take-home, Product Intern

yadavchandan6103@gmail.com

2026-10-24

App: https://oauth-setup-guide-agent.vercel.app

API health: https://oauth-setup-guide-agent-2.onrender.com/api/health

Repo: https://github.com/yadavchandan84/oauth-setup-guide-agent

Problem understanding

Connecting an AI agent (or any product) to a third-party app over OAuth is rarely the hard part. The hard part is everything *around* the token: registering an app in a developer console, picking the right scopes, wiring redirect URIs, and — for many platforms — passing a review or verification process before the integration can serve real users.

That approval surface is wildly inconsistent across providers:

- **GitHub** gives you a working OAuth app in minutes with no review gate.
- **Google Workspace** can require OAuth verification plus a CASA security assessment that takes weeks when restricted scopes are involved.
- **Shopify** needs a Partner account, an App Store review, and a *separate* protected-customer-data approval for public apps.

An engineer wiring up a new integration has to rediscover each platform's rules from scratch, often scattered across docs that move, get renamed, or bury the approval timeline. The cost shows up as surprised deadlines ("we can't launch, Google verification is still pending") and over-broad scope requests that make review harder.

The job to be done: tell me, for a given app, *how hard* the OAuth approval is, *what I have to do*, *what an agent can do for me*, *how long approval takes*, and *where the landmines are* — with each claim traceable to an official source, so I can plan before I write a line of code.

This project answers that for 14 popular apps as a curated, verified library, and can generate the same structured guide live for any app the user names.

Scope decision and rationale

What this is

A full-stack **OAuth Approval Guide Agent**: a curated library of 14 verified guides plus a live agent that generates a guide for any app on demand. Every guide uses one schema, so curated and live output are structurally identical and directly comparable.

Why a curated library *and* a live agent

A take-home that only generated guides live would be slow and fragile in front of a reviewer: an agent that searches docs and calls an LLM takes 30–90 seconds, depends on third-party uptime, and costs money per run. A library that was *only* curated would not demonstrate the agent at all.

The chosen split gets both:

- The **frontend bundles the 14 curated guides**, so the library is fully browsable with zero backend calls — it works instantly even while the free Render backend is cold.
- The **live agent** is a real, streaming pipeline the reviewer can trigger, with progress shown step by step over Server-Sent Events so the wait reads as intentional.

Deployment shape

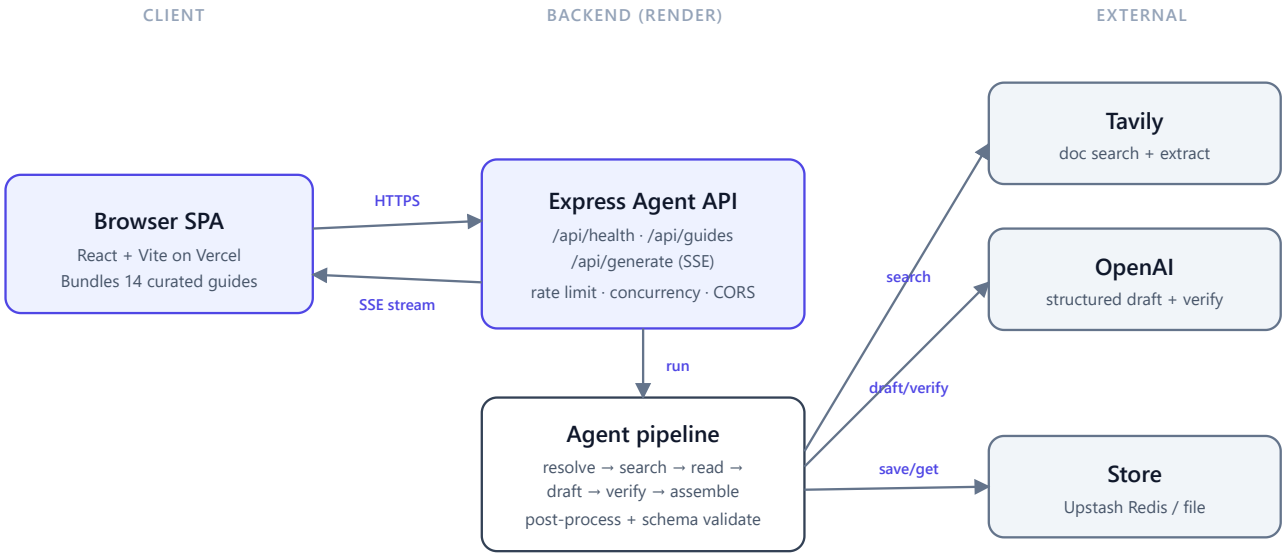
- **Frontend → Vercel** (React + Vite SPA). One static deploy, no cold start.
- **Backend → Render** (Node + Express). A long-lived server has no serverless timeout, so a 90-second agent run streams cleanly. The free tier sleeps after inactivity; the README documents an UptimeRobot warm-ping and a pre-submission wake-up.

Explicit non-goals

- **No token exchange / no acting on the user's behalf.** The agent explains *how* to get a token and what a human must do; it never performs the OAuth flow or stores user credentials. This keeps the security surface small and the scope honest.
- **No logo hot-linking.** Guides use a monogram and brand color, not trademarked image assets.
- **No invented facts.** A section is marked `verified` only when an official URL that passes the link checker backs it. Approval-wait figures that no official page states are marked `community`, and genuinely open questions are marked `uncertain`.
- **No video walkthrough.** The submission is this PDF plus the live links.

How it works

The system is three deployable packages plus a shared schema.



Frontend (Vercel)

A React + Vite single-page app. It **bundles all 14 curated guides at build time**, so browsing the library, reading a guide, and deep-linking (e.g. `/guides/github`) needs no backend. It talks to the backend only for live generation and for permalinks to live-generated guides.

Backend (Render)

A Node + Express API with three surfaces:

- `GET /api/health` — a dependency-free liveness check (also what UptimeRobot pings).
- `GET /api/guides` and `GET /api/guides/:slug` — curated and recently-generated guides.
- `POST /api/generate` — the live agent, streamed as Server-Sent Events.

Because it is a normal long-lived server (not a serverless function), a 90-second agent run streams without hitting a platform timeout.

The live agent pipeline

A single `POST /api/generate` request drives seven stages, each emitting a `step` event so the UI can show progress:

1. **resolve** — normalize the input (app name or docs URL) to a stable key; a curated or cached hit short-circuits here.
2. **search** — three parallel Tavily queries over official docs.
3. **read** — Tavily extract on the top pages.
4. **draft** — OpenAI structured output produces a guide draft against a strict schema.
5. **verify** — a second OpenAI pass checks the draft against the retrieved sources and lowers confidence where support is thin.
6. **assemble** — deterministic post-processing: normalize, allow-list console URLs, cap confidence, validate against the schema.
7. **save** — persist for a shareable permalink.

Shared schema and sync

`shared/schema.ts` (a zod schema) and the 14 guide JSON files are the single source of truth. A dependency-free script, `scripts/sync-shared.mjs`, copies them into each app before every build, dev, and test run, so the frontend and backend compile

against exactly the same types and data without cross-package module resolution.

Safety posture

The server never fetches a user-supplied URL directly (Tavily does the fetching). Retrieved page content is treated as untrusted input in prompts. LLM output is schema-validated and React-escaped, never injected as raw HTML. Keys live only in platform environment variables.

Guide format

Every guide — curated or live — is the same validated structure. That uniformity is what makes apps comparable and makes the live agent's output trustworthy: it has to fill the same slots the curated guides do.

Sections

- **App + brand** — name, category, website, a monogram and brand color (no hot-linked logos).
- **OAuth availability** — supported / not-supported / unclear, with alternatives.
- **Difficulty** — `Easy` · `Moderate` · `Hard` · `Very Hard`, with a one-line reason.
- **Time estimate** — hands-on effort and approval wait as `{min, max, unit}` ranges, plus the critical path.
- **Review** — whether approval is required, when, which review types apply, and the ordered stages that drive the approval timeline.
- **Prerequisites** — what must exist before you start, each tagged with an actor.
- **Steps** — ordered actions with detail, an optional console URL, an actor, an estimate, and a gotcha.
- **Redirect URIs** — notes and example callback URLs.
- **Scopes** — recommended scopes with purpose, plus sensitive/restricted scopes and their consequences.
- **Blockers** — common failure → symptom → cause → resolution.
- **Agent handoff** — what the agent can do, what a human must do, and the handoff points.

Actors

Every step, prerequisite, and blocker carries an **actor** so the agent/human split is explicit:

- **Agent can do** — automatable (writing the callback route, implementing the token exchange).
- **You must do** — requires a human (signing in, generating a secret, submitting for review).
- **Agent can assist** — a human decision the agent can draft or prepare.

Confidence and evidence

Each of the nine content sections carries a **confidence level** and source references:

- **verified** — backed by an official source URL that passed the link checker.
- **community** — reported by forums/blogs or widely-known, but not stated on an official page (typical for approval-wait figures).
- **uncertain** — confirm before relying on it.

Confidence is never inflated: the live agent's post-processing caps a section's confidence at the verifier's judgment, and a section can only be `verified` when an official source actually supports it.

Sample guides

Three curated guides, rendered directly from the real JSON in `shared/guides/`, chosen to span the difficulty range: **GitHub** (Easy, no review), **Google Workspace** (Very Hard, verification + security assessment), and **Shopify** (Hard, App Store review plus protected-customer-data approval).

App	Difficulty	Hands-on	Approval wait	Review
GitHub	Easy	10–20 minutes	none	none
Google Workspace / Gmail	Very Hard	1–3 days	4–12 weeks	oauth-verification, security-assessment
Shopify	Hard	1–3 days	1–4 weeks	partner-program, app-review

GitHub — Easy

GitHub OAuth apps are the fastest OAuth setup of the curated set: register an app in your developer settings, set the callback URL, and start the authorization code flow immediately. There is no app-review or approval gate, so an app works the moment it is registered. GitHub recommends GitHub Apps (fine-grained permissions, short-lived tokens) over classic OAuth apps for most new integrations.

First steps

1. Open Developer settings and start a new OAuth app You must do
2. Fill in the application details Agent can assist
3. Choose device flow and token expiration, then register You must do
4. Store the client ID and secret, then implement the flow Agent can do

Top blockers

- *redirect_uri_mismatch* — Add the exact callback URL (scheme, host, path) to the app's Authorization callback URLs and retry.
- *Token unexpectedly expires* — Implement the refresh token exchange, or disable token expiration only if your code cannot yet refresh tokens.

Agent / human split

- Agent can do Implement the authorization code exchange and token refresh
- Agent can do Draft the callback route and scope list
- You must do Sign in to GitHub and register the OAuth app
- You must do Generate and securely store the client secret

Sources

- Creating an OAuth app - GitHub Docs (official)
- Authorizing OAuth apps - GitHub Docs (official)
- Scopes for OAuth apps - GitHub Docs (official)

Google Workspace / Gmail — Very Hard

Creating OAuth credentials in Google Cloud is quick, but shipping a public app that touches Gmail, Drive, or Calendar user data is the hardest approval path in this set. Sensitive scopes trigger Google's OAuth app verification; restricted scopes (such as Gmail message bodies) additionally require an independent CASA security assessment and annual re-verification. Plan for weeks, not days, before a public app leaves the unverified state.

First steps

1. Create OAuth client credentials in Google Cloud Agent can assist
2. Configure the OAuth consent screen You must do
3. Select the minimum scopes and justify them You must do
4. Submit for OAuth app verification You must do

Top blockers

- *Unverified app warning blocks users* — Complete OAuth app verification (and CASA for restricted scopes), or restrict the app to test users while in development.
- *CASA assessment rejected or stalled* — Address the assessor's findings and resubmit; budget several weeks and plan for annual re-verification.

Agent / human split

- Agent can do Create the OAuth client and wire the token exchange and refresh

- Agent can do Draft scope justifications and the list of narrowest scopes
- You must do Own the OAuth consent screen, branding, and privacy policy
- You must do Submit for verification and respond to Google's questions

Sources

- Using OAuth 2.0 to Access Google APIs (official)
- Gmail API OAuth 2.0 scopes (official)
- OAuth App Verification Help Center (official)
- CASA - App Defense Alliance (official)

Shopify — Hard

A Shopify app authenticates to the GraphQL Admin API with an access token obtained after a merchant installs the app and approves its scopes; the CLI and app templates handle most of the OAuth flow for you. Building and testing is quick, but a public app distributed through the Shopify App Store goes through a Partner Program setup and an app review, and features that read customer personal data need a separate protected-customer-data approval.

First steps

1. Create the app and declare access scopes Agent can assist
2. Install to a dev store and obtain a token Agent can do
3. Request protected customer data access if needed You must do
4. Prepare the listing and submit for App Store review You must do

Top blockers

- *App Store submission rejected* — Address the reviewer feedback against the App Store best practices and resubmit.
- *Customer-data fields return empty or error* — Request protected customer data access; for custom apps, verify the store plan's field access.

Agent / human split

- Agent can do Scaffold the app with Shopify CLI and wire the token flow
- Agent can do Declare scopes and implement Admin API calls with the access token
- You must do Create and own the Partner account and app
- You must do Request protected customer data access and submit for App Store review

Sources

- About app authentication - Shopify (official)
- Shopify App Store best practices (official)

Live agent flow

When a reviewer types an app name the library doesn't already cover and clicks **Generate**, the browser opens a streaming `POST /api/generate` request and renders progress from Server-Sent Events in an `aria-live` activity timeline.

What the reviewer sees

A sequence of step events, using the labels the pipeline emits:

1. *Resolving Asana…*
2. *Searching docs…* (three Tavily queries in parallel)
3. *Reading <page title>…* (one step per page read; source events stream in alongside)
4. *Drafting guide…* (the draft is assembled and schema-validated inside this step, with at most one repair attempt)
5. *Verifying claims…*
6. *Saving shareable guide…*, which ends as *Saved shareable guide*, then the finished guide renders in place.

Streaming the steps is deliberate: a 30–90 second wait reads as the agent working, not as a hung page.

Short-circuits and guards

- **Curated or cached hit** resolves in the first stage and returns immediately, flagged `cached`. These do **not** count against the rate limit.
- **No keys configured** → a `503` JSON response with code `CONFIG_MISSING`, sent before any stream opens. The UI shows a clear message instead of a blank screen, and the curated library keeps working.
- **Rate limit** → `429` with a `Retry-After` so a reviewer (or a bot) can't run up the API bill.
- **Concurrency cap** → `503 BUSY` when too many generations run at once.
- **Timeout** → the run aborts after `GENERATION_TIMEOUT_MS` (code default 120 seconds; the Render blueprint sets 180 seconds) with a `TIMEOUT` error event.
- **Client disconnect** → the server detects the closed response and stops the pipeline (`CLIENT_GONE`), so an abandoned run doesn't keep spending.

Determinism after the model

The LLM only ever produces a *draft*. Everything that makes a guide trustworthy happens in deterministic post-processing afterward: console URLs are allow-listed to the app's own domains, confidence is capped at the verifier's judgment, foreign or invented URLs are dropped, and the result must pass the same `GuideSchema` the curated guides pass — or the run fails loudly rather than shipping a malformed guide.

Use of agents

Agents were used in two distinct ways: **inside the product**, and **to build it**.

Agents inside the product

The live generation pipeline is an agent in the literal sense: given only an app name or docs URL, it decides what to search, reads the results, drafts a structured guide, and runs a second verification pass that critiques its own draft against the retrieved sources. It is deliberately *bounded* — it searches and reads, but it never fetches arbitrary user URLs itself, never performs the OAuth flow, and never ships output that fails schema validation. The model proposes; deterministic code disposes.

Agents to build the project

Development was done with an AI coding assistant under a planned, reviewed workflow rather than ad-hoc prompting:

- The work was decomposed into a design document and a set of discrete features, each with its own acceptance criteria and verification commands.
- Each feature was implemented against those criteria, then checked by running the project's real build, type-check, and test suites — not by eyeballing output.
- **Curated content was researched, not hallucinated.** Every cited URL was actually retrieved during research before being used to back a claim, and the link checker re-confirms all of them. Where an official page did not state a figure (approval wait times, for example), the section is marked `community` rather than invented as `verified`.

Honesty over polish

The guiding rule throughout: a confident-sounding wrong answer is worse than an honest "uncertain." That shows up in the schema (three confidence levels, source references per section), in the pipeline (confidence capping, URL allow-listing), and in the content (open questions logged as `uncertain` instead of smoothed over).

Evidence and verification log

Every curated guide was researched against official documentation on **20 October 2026**, and every cited URL is re-checked by `scripts/check-links.mjs`. The table below is generated from the committed `shared/link-check-report.json`.

For curated guides, "confirmed" or `verified` means two things: the cited URL resolves, and the claim matched the page as read on 2026-10-20. The GitHub, Jira, Linear and Shopify guides were re-read against their cited pages on 2026-10-23.

Status	Count
ok	46
redirected	4
blocked	0
broken	0

Redirected / blocked URLs (manually checked, reachable):

URL	Status	Final URL
https://github.com/settings/developers	redirected	https://github.com/login?return_to=https%3A%2F%2Fgithub.com%2Fsettings%2Fdevelopers
https://slack.com	redirected	https://slack.com/intl/en-in/
https://stripe.com	redirected	https://stripe.com/in
https://www.salesforce.com	redirected	https://www.salesforce.com/in/?ir=1

Learned (docs that have moved or been renamed)

- **Slack** — developer docs moved from `api.slack.com` to `docs.slack.dev`; guides cite the current URLs.
- **Stripe** — docs moved from `stripe.com/docs` to `docs.stripe.com`.
- **Discord** — developer docs moved from `discord.com/developers/docs` to `docs.discord.com/developers`.
- **HubSpot** — OAuth pages now live under a `legacy-apps/` path, reflecting the move to the newer projects platform. The new-vs-legacy flow for a brand-new app is logged as `uncertain`.
- **Microsoft** — "Azure AD" is now "Microsoft Entra ID"; guides cite the `learn.microsoft.com/entra` identity-platform URLs.
- **Google** — the OAuth verification help page is now at `support.google.com/cloud/answer/13463073`.

Corrected (claims changed after reading the source)

- **Figma** — the review question was *resolved*, not left open. The official OAuth-apps page states that public OAuth apps are reviewed by Figma on publish and can only authorize users after approval (private/draft apps are not reviewed). The guide was switched to `review.required: true`, review type `app-review`, with a publish-review stage; the exact turnaround is unpublished and stays `community`.
- **Website redirects normalized** — Notion (`www.notion.so` → `www.notion.com`), Zoom (`zoom.us` → `www.zoom.com`), and Microsoft (`learn.microsoft.com` → `learn.microsoft.com/en-us/`) app websites were updated to their final URLs.

Unverified / treated as community or uncertain

- **Approval-wait durations** for review processes (Google CASA, Shopify App Store, Zoom, Microsoft publisher verification) are **community-reported** — providers rarely publish fixed SLAs — so those figures are never marked `verified`.
- **Author-estimated hands-on times** are `community`.
- **Salesforce** — `developer.salesforce.com` returns HTTP 403 to automated fetches (bot protection) but is reachable to a browser-like client and to the link checker; it is cited as official with that caveat recorded.
- A handful of report entries show as **redirected** rather than broken and are intentionally left as-is: a GitHub console URL behind a login gate, and region/locale geo-redirects (Slack, Stripe, Salesforce) that differ per reviewer region, where "correcting" to a geo-specific URL would itself be wrong.

Limitations

- **Approval timelines are estimates.** Most providers don't publish review SLAs, so wait-time figures are community-reported ranges, marked as such. Treat them as planning aids, not guarantees.

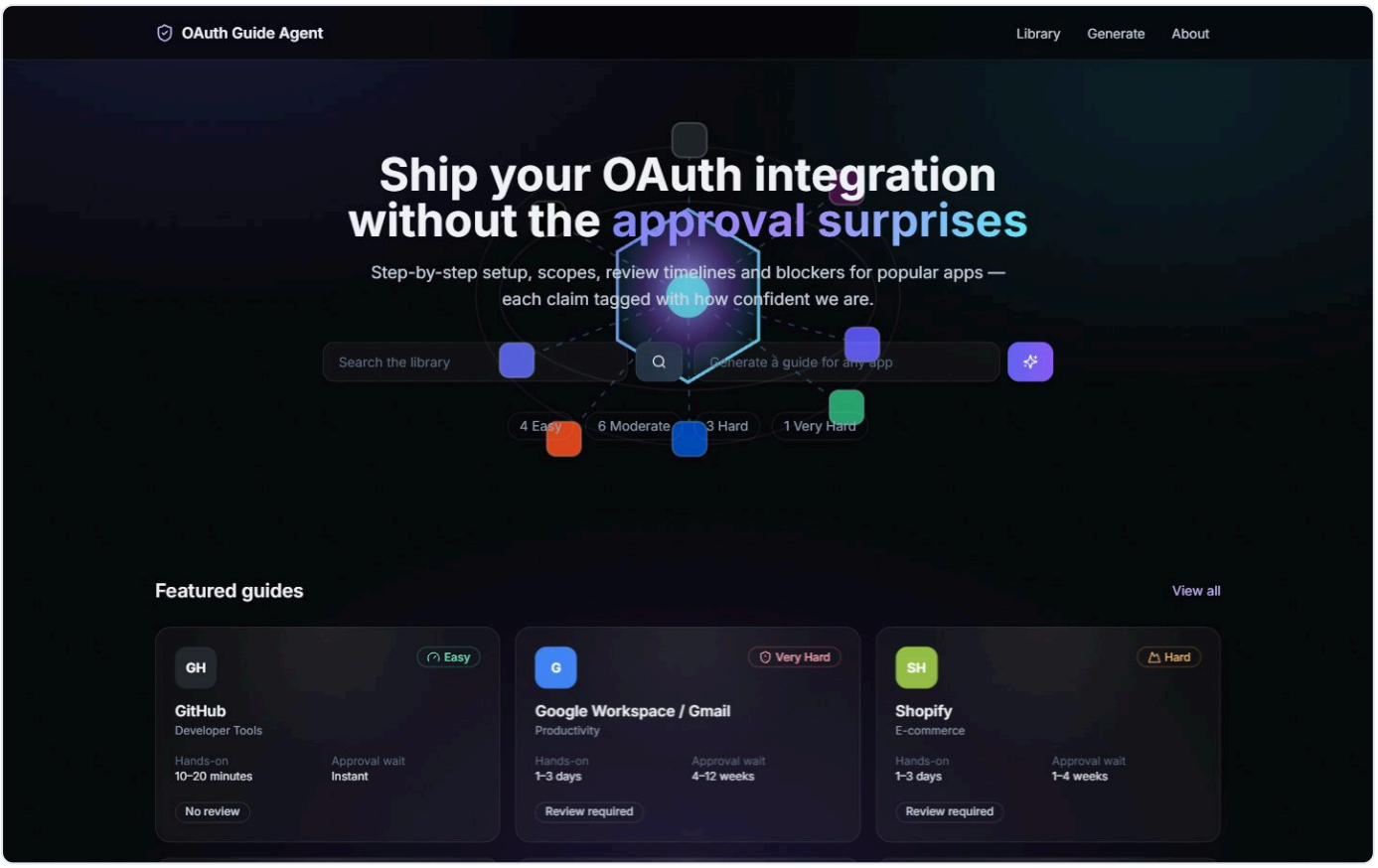
- **Docs drift.** OAuth consoles and documentation move and get renamed often (several already did during research — see the evidence log). A guide is a snapshot as of its `lastVerified` date; the link checker catches dead URLs but not silent content changes.
- **Live generation depends on third parties.** A live run needs Tavily and OpenAI reachable and keyed. Without keys the endpoint returns a clean `503`, and the curated library still works — but the live feature itself is only as reliable as those services.
- **Free-tier cold starts.** The Render free tier sleeps after inactivity, so the first request after idle can take 30–60 seconds. The README documents an UptimeRobot warm-ping and a pre-submission wake-up; a paid tier removes this entirely.
- **Ephemeral storage without Upstash.** On Render's free tier the file store is lost on every redeploy, on restart, and when the service spins down after about 15 idle minutes. Without Upstash, a live-guide link can break after a period of inactivity; an UptimeRobot ping reduces spin-downs but does not remove restarts or redeploys. Durable, shareable links require the optional Upstash Redis configuration.
- **The keyed live path is tested with stubs.** Real OpenAI and Tavily calls are covered by tests with stubbed providers only. A real end-to-end run happens after deploy, with the author's keys.
- **The agent plans, it does not act.** It explains how to obtain a token and what a human must do; it never performs the OAuth flow, exchanges a code, or stores user credentials. That is a deliberate scope boundary, not an oversight.
- **Curated coverage is 14 apps.** Anything outside that set goes through live generation, which is slower and carries the caveats above.
- **Accessibility is strong but not formally audited.** The UI uses semantic landmarks, keyboard-operable controls, `aria-live` regions, and contrast targets, but full WCAG conformance needs manual assistive-technology testing that is out of scope here.

Future work

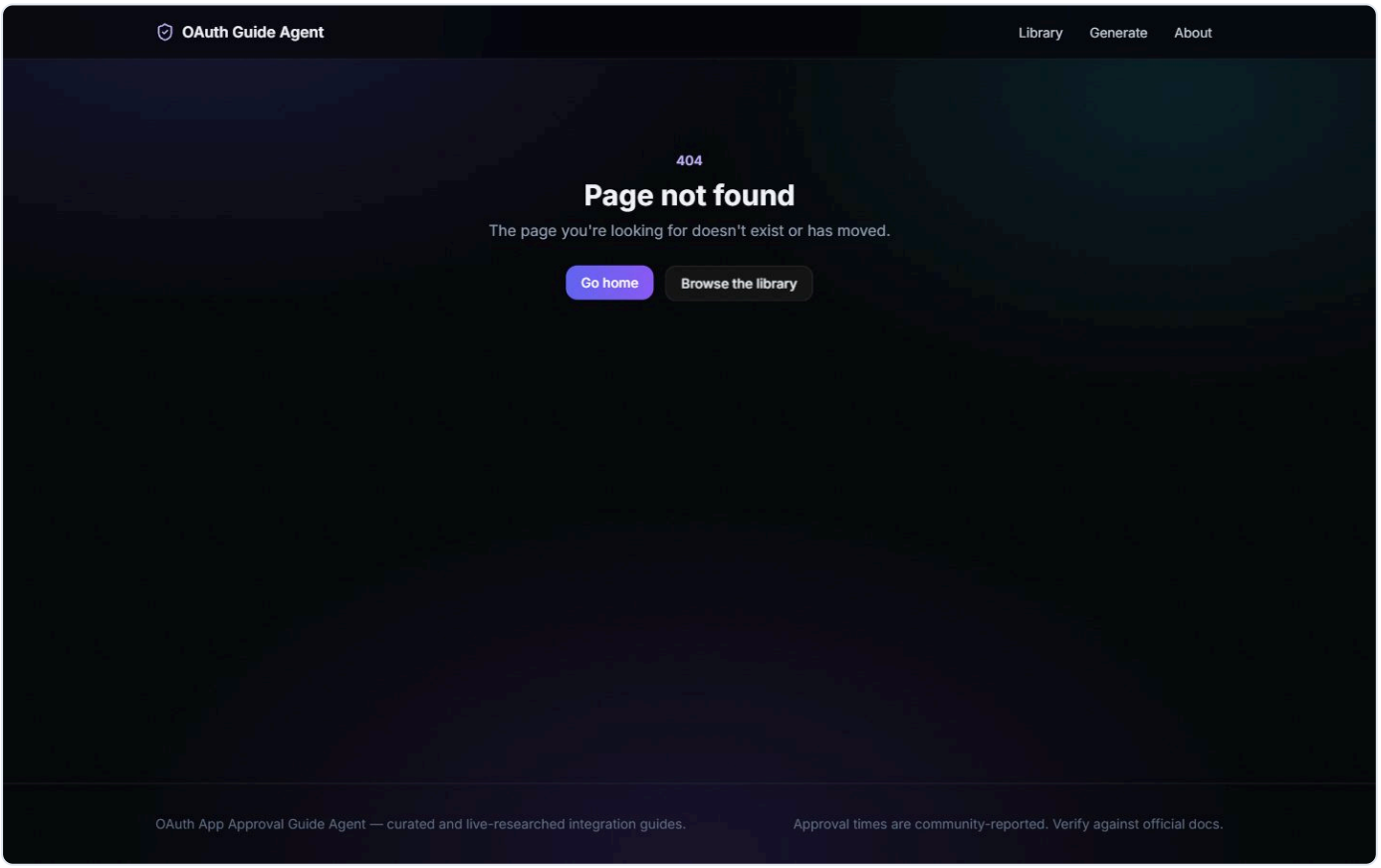
- **Scheduled re-verification.** Run the link checker on a schedule and flag guides whose sources moved or broke, so `lastVerified` stays honest without manual sweeps.
- **Diff-aware updates.** Detect when an official page's content changes (not just its URL) and re-run the relevant section through the agent.
- **Expand curated coverage.** Promote the most-generated live apps into the curated, hand-verified set.
- **Scope advisor.** Given the feature a developer wants, recommend the narrowest scope set and warn when a requested scope triggers a heavier review path (e.g. Google restricted scopes → CASA).
- **Agent-executable handoffs.** For the steps already tagged "agent can do," emit copy-paste-ready snippets (callback route, token exchange) tailored to the user's stack.
- **Durable links by default.** Ship with Upstash wired so every live-generated guide has a permanent shareable URL out of the box.
- **Webhook/notification integration.** Let a user subscribe to a guide and get notified when its verification status changes.

Screenshots

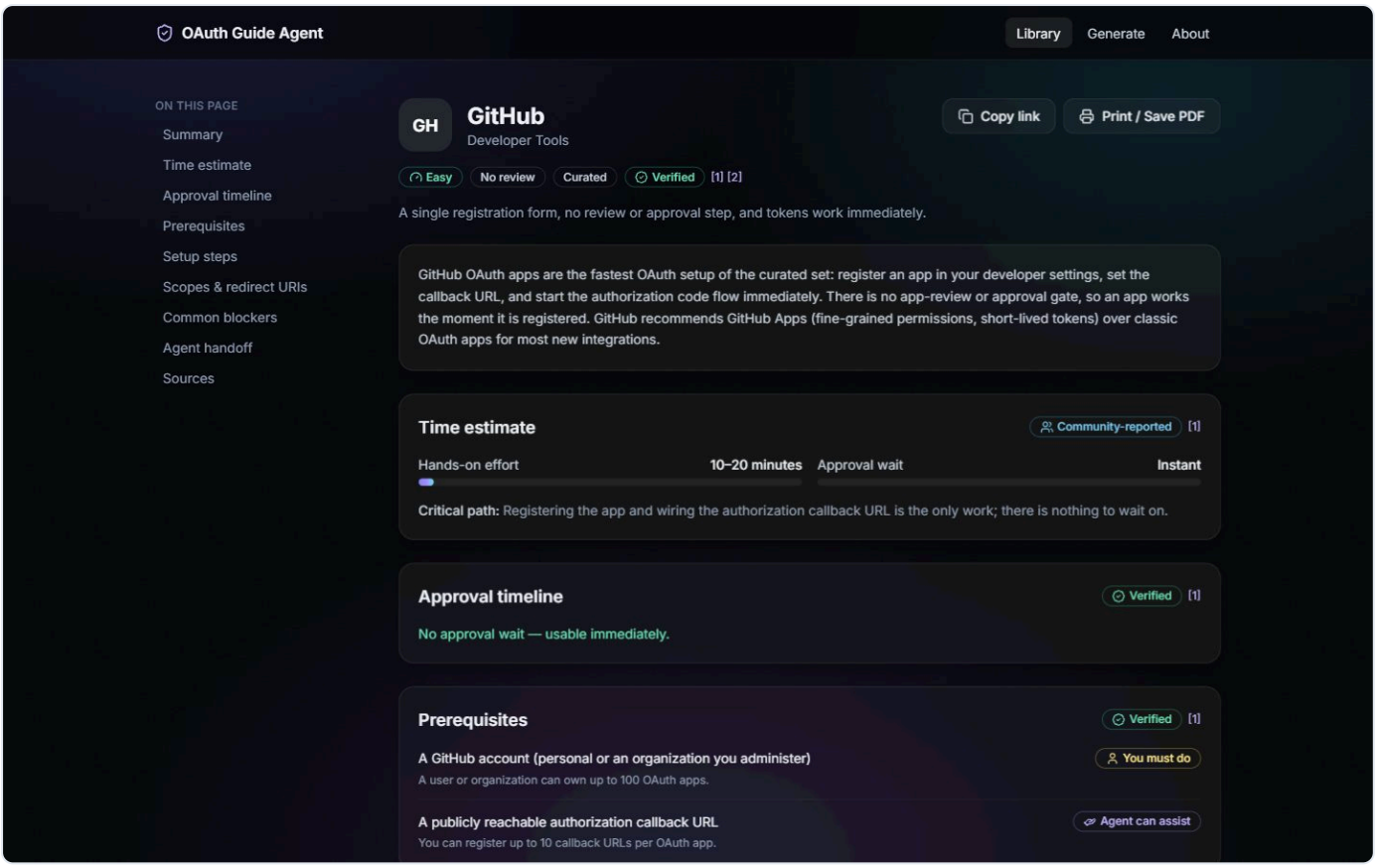
Captured from the running app against a local `vite preview` build. The WebGL hero renders via SwiftShader in headless capture; on a real GPU it animates.



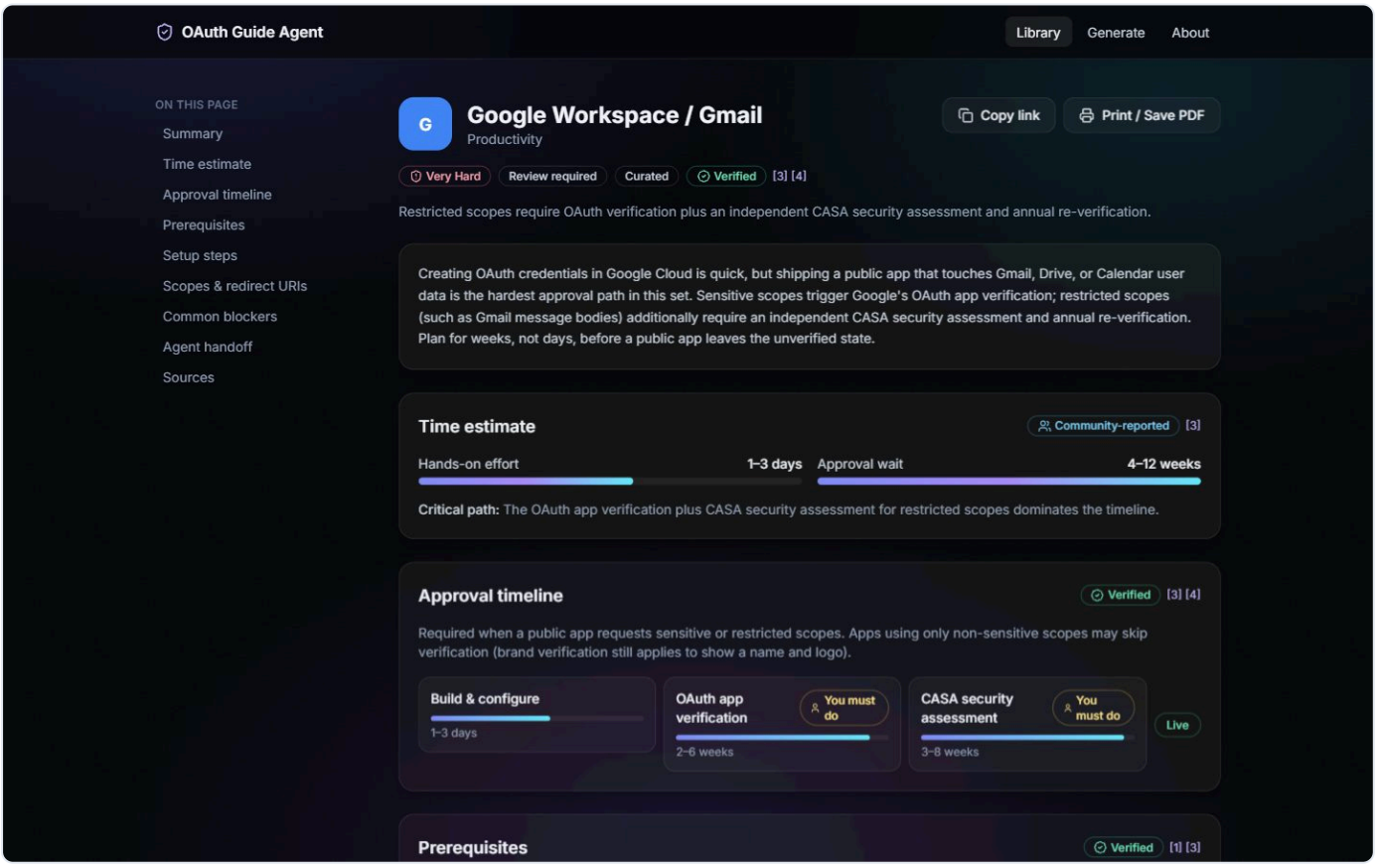
Landing page with the 3D hero.



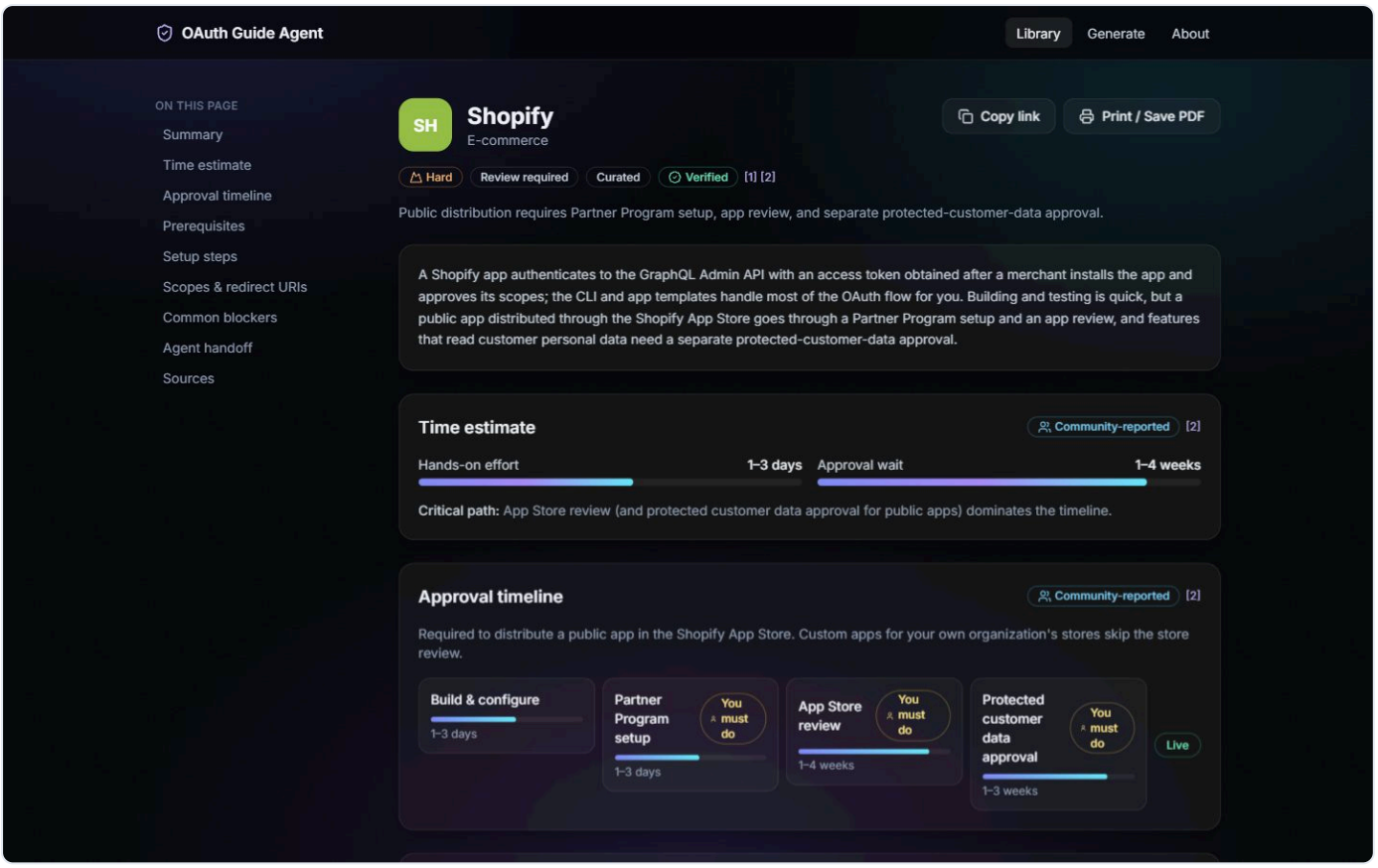
Curated guide library.



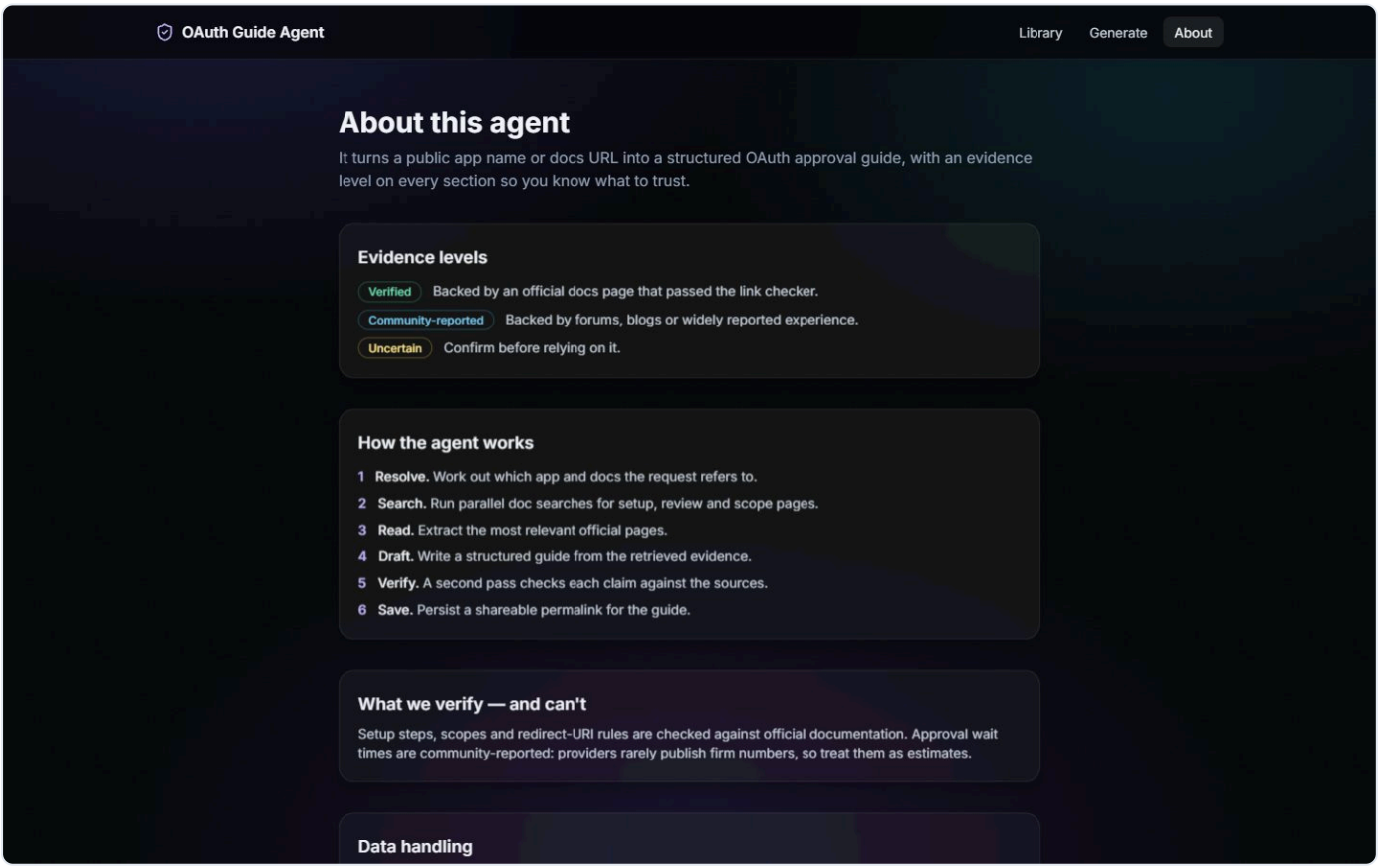
GitHub guide detail (Easy, no review).



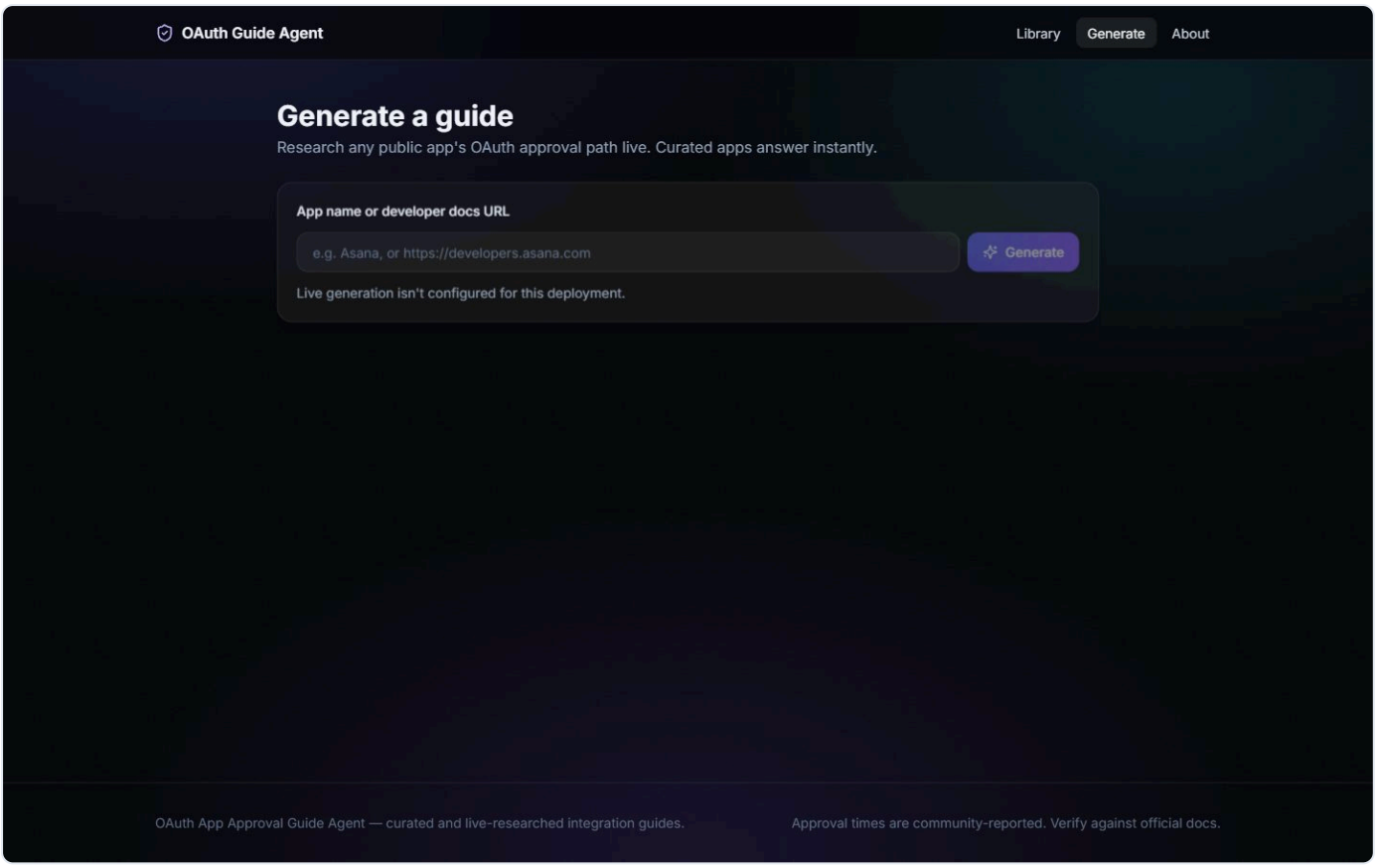
Google Workspace guide (Very Hard, verification + security assessment).



Shopify guide approval timeline.



About page explaining evidence levels.



Live generation form, idle state.